

NewsFlux Data Ingestion Analyse & Uitbreidingsplan

Executive Summary

Het NewsFlux pipeline is een goed opgezette multi-source AI news intelligence engine die data ingest vanuit 8+ bronnen, verwerkt via TF-IDF en DuckDB, en uiteindelijk een aitooldradar.json genereert die het nuchter.ai / debesteaitools.nl Astro-platform voorziet van live stats. De huidige ingestie-architectuur is solide maar mist een aantal gerichte databronnen en gedetailleerde veldverrijking die de tool scores significant kunnen verbeteren.

1. Huidige Ingestie Architectuur

1.1 Data Flow (eind-tot-eind)

De volledige pipeline volgt 4 stappen:[1][2]

1. **Ingest** (src/ingest.py) — Haalt ruwe data op uit alle bronnen en slaat op in DuckDB raw-tabellen (raw_inoreader, raw_twitter, raw_reddit, raw_linkedin, etc.)
2. **Trend Intelligence** (src/trendintelligence.py) — Maakt dagelijkse snapshots met gewogen signalen, berekent velocity/acceleration, classificeert lifecycle phases
3. **Radar Generator** (src/generate_aitoolsradar.py) — Aggregeert trendsnapshots per tool-slug, berekent buzz score (0–100), en exporteert aitooldradar.json
4. **Sync naar Frontend** (scripts/sync-tools-content.ts) — ETL die aitooldradar.json leest, nieuwe tool-JSON-bestanden aanmaakt als draft, en enrichmentdiff.json genereert

1.2 Actieve Bronnen

Bron	Script	Gewicht	Tabellen
Inoreader/RSS	ingest.py	1.0x (nieuwsartikelen)	articles_flat
Twitter/X	ingest.py (TwitterAPI.io)	1.0x baseline	twitter_flat
Reddit	ingest.py (PRAW)	1.0x	reddit_flat
HackerNews	hackernewsfetcher.py	3.0x developer consensus	articles_flat (via RSS)
ArXiv	arxivfetcher.py	4.0x research frontier	trendsnapshots.source_arxiv
GitHub Trending	githubtrending.py	EXPERT signal	github_trending_snapshots
Bluesky	ingest.py (AT Protocol)	2.5x tech/AI community	trendsnapshots.source_bluesky
Product Hunt	Via RSS/Inoreader	2.5x product-market fit	Via articles_flat
LinkedIn	ingest.py	1.0x	linkedin_flat
Newsletters	Inoreader labels	1.0x editorial	articles_flat

1.3 Huidige Buzz Score Formule

De buzz score (0–100) wordt berekend in `generate_aitoolsradar.py` als gewogen som:[1]

Component	Gewicht	Bron
Mention count	30%	trendsnapshots.mention_count
Source diversity	15%	distinct source types
Velocity/groei	20%	mentions/dag trend
GitHub stars	15%	github_trending_snapshots
Reddit posts	5%	reddit_flat + Reddit RSS
Product Hunt	5%	PH RSS via Inoreader
Newsletter mentions	5%	curated editorial sources
Expert ratio	5%	HN/ArXiv/expert mentions ratio

2. Wat Wordt Nu Geïngesteerd per Bron

2.1 trendsnapshots Tabel — Kerndata

Elke snapshot-rij per (snapshot_date, keyword) bevat:[1]

```

snapshot_date  DATE
keyword        VARCHAR
mention_count  INTEGER
source_twitter INTEGER
source_reddit  INTEGER
source_articles INTEGER
source_linkedin INTEGER
source_github  INTEGER -- V2
source_hackernews INTEGER -- V2
source_arxiv   INTEGER -- V2
source_bluesky INTEGER -- V2
source_count   INTEGER
avg_engagement FLOAT

```

```
expert_mentions  INTEGER
mainstream_mentions INTEGER
sample_headlines  JSON
related_terms     JSON
```

2.2 aitooldradar.json — Geëxporteerde Velden

Het radar JSON-bestand (gelezen door de Astro frontend via `tools-engine.ts`) bevat per tool:[2]

Veld	Type	Afkomstig van
slug	string	tool taxonomie
buzz_score	int (0–100)	berekend in generator
trend_direction	rising/stable/declining	velocity drempel
mentions	int	aggregaat trendsnapshots
velocity	float	exp. gewogen dagelijkse delta
acceleration	float	2e afgeleide velocity
source_count	int	aantal unieke brontypen
sources	string[]	lijst actieve bronnen
phase	string	lifecycle classificatie
days_active	int	eerste → laatste mention
history	[{date, mentions}]	dagelijkse tijdreeks
github_stars	int	trending repos stars
github_repos	int	aantal trending repos
github_top_repos	array	top 3 repos met beschrijving
reddit_posts	int	Reddit post count
product_hunt_mentions	int	PH RSS mentions
newsletter_mentions	int	curated newsletter count
source_articles	int	raw artikel count

2.3 RadarTool Schema (sync-tools-content.ts)

Het TypeScript ETL-script leest van `aitoolsradar.json` en verwacht naast statistieken ook een `enrichment` object met:[2]

```
type RadarEnrichment = {  
  description_long_nl?: string  
  best_for_nl?: string  
  use_cases?: string[]  
  strengths?: string[]  
  limitations?: string[]  
  pricing_nl?: string  
  open_source?: boolean  
  enriched_at?: string  
}
```

Dit enrichment-object wordt **nu gegenereerd door Perplexity** via het premium pipeline (`enrich_digest_items.py`), maar is **nog niet geïntegreerd in de radar generator** voor tool-specifieke enrichment.

3. Geïdentificeerde Gaps & Kansen

3.1 Missende Data in `aitoolsradar.json`

De volgende velden zijn al deels berekend in de backend maar **worden nog niet geëxporteerd** naar het frontend JSON:[2][1]

Ontbrekend Veld	Beschikbaar In	Impact
expert_ratio	trendsnapshots	Onderscheidt niche vs. mainstream tools
mainstream_ratio	trendsnapshots	Idem
peak_mentions	generate_aitoolsradar.py	Historische piek zichtbaar maken
first_seen	trendsnapshots	"Nieuw" badge nauwkeuriger
last_seen	trendsnapshots	Staleness detectie
cross_source_headlines	sample_headlines	Concrete citaten voor tool-pagina's
related_terms	trendsnapshots.related_terms	"Gerelateerde tools" sectie
acceleration	berekend maar niet geëxporteerd	Vroege signalen versnellende tools
narrative_summary	trend_narratives_v2	LLM-gegenereerde duiding per tool
forward_signal	trend_narratives_v2	Voorspelling toekomstige adoptie

3.2 Bron-gaps: Wat Nog Niet Wordt Geïngesteerd

Bron	Waarde	Implementatie Suggestie
YouTube	1.5x gewicht al gedefinieerd maar niet actief als eigen pipeline	YouTube Data API v3 voor AI channel mentions per tool
Product Hunt API	Nu via RSS (beperkt)	Directe PH API voor upvotes, comments, launch datum
npm / PyPI downloads	Developer adoptie metric	Weekelijkse download counts per tool SDK
Chrome Web Store	Extension-adoptie	Install count + rating voor browser-gebaseerde tools
Discord activity	Community grootte	Public server member counts (geen auth nodig)
Stack Overflow	Developer vraagstelling	Questions per tool per maand via SO API
Lobste.rs / dev.to	Developer community	RSS feeds gratis beschikbaar

3.3 Enrichment Pipeline — Niet Gekoppeld aan Tools

De premium enrichment pipeline (`enrich_digest_items.py`) genereert rijke metadata via Perplexity per nieuws-item, maar **dit is nog niet gekoppeld aan de tool-specifieke enrichment** die `sync-tools-content.ts` verwacht. Het enrichment veld in RadarTool (`description_long_nl`, `strengths`, `limitations`, etc.) wordt momenteel **niet automatisch gevuld** vanuit de pipeline — dit moet handmatig of via Keystatic CMS gedaan worden.[1]

4. Concrete Aanbevelingen

4.1 Kort Termijn: Exporteer Bestaande Data

De snelste winst zit in het uitbreiden van `generate_aitoolsradar.py` om velden die al worden berekend ook te exporteren. Voeg toe aan het output-object per tool:[1]

```
# In generate_aitoolsradar.py — exporteer al berekende velden
tool_entry = {
    ...bestaande velden...,
    "expert_ratio": round(tool_stats["expert_mentions"] / max(total, 1), 2),
    "mainstream_ratio": round(tool_stats["mainstream_mentions"] / max(total,
    "peak_mentions": tool_stats["peak_mentions"],
    "first_seen": tool_stats["first_seen"],
    "last_seen": tool_stats["last_seen"],
    "acceleration": tool_stats["acceleration"],
    "related_terms": tool_stats["sample_headlines"][:3], # al beschikbaar
}
```

Impact: Geen nieuwe API-aanroepen, direct bruikbaar voor TrendCard componenten.

4.2 Middel Termijn: Tool-specifieke Enrichment Automatiseren

Maak een nieuwe Python-script `enrich_tool_profiles.py` dat per tool in de taxonomie:[1]

1. Perplexity aanroept met "Wat is {tool_name}? Geef: beschrijving (NL), sterke punten, beperkingen, pricing model"
2. Output cached in `data/enrichments/tools/{slug}.json`
3. `generate_aitoolsradar.py` laadt deze cache en voegt toe als enrichment object
4. `sync-tools-content.ts` pikt het op in het bestaande ETL-process

Dit sluit aan op het bestaande `RadarEnrichment` type in `sync-tools-content.ts`.:[2]

4.3 Middel Termijn: YouTube Pipeline Activeren

De SmartSourceScanner in trendanalysis.py kent al een 1.5x gewicht voor YouTube, maar er is **geen actieve YouTube-fetcher**. Minimale implementatie:[1]

```
# src/youtubefetcher.py
from googleapiclient.discovery import build

def fetch_youtube_mentions(tool_slug: str, api_key: str) -> dict:
    youtube = build('youtube', 'v3', developerKey=api_key)
    request = youtube.search().list(
        q=tool_slug, type='video', order='relevance',
        publishedAfter=(datetime.now() - timedelta(days=14)).isoformat() + 'Z',
        maxResults=10
    )
    response = request.execute()
    return {
        "mention_count": len(response.get('items', [])),
        "total_views": sum(int(i['statistics'].get('viewCount', 0))
                           for i in response['items']),
    }
```

4.4 Lang Termijn: npm/PyPI Developer Adoptie Signal

Voor tools zoals LangChain, LlamaIndex, n8n, CrewAI en andere die SDK's hebben, is het **download volume op npm/PyPI een objectiever adoptiemaat** dan social mentions:[1]

```
# npm: gratis, geen auth
import requests

def get_npm_downloads(package: str, period: str = "last-month") -> int:
    r = requests.get(f"https://api.npmjs.org/downloads/point/{period}/{package}")
    return r.json().get("downloads", 0)

# PyPI: ook gratis
def get_pypi_downloads(package: str) -> int:
```

```
r = requests.get(f"https://pypistats.org/api/packages/{package}/recent")
return r.json().get("data", {}).get("last_month", 0)
```

Voeg `npm_downloads` en `pypi_downloads` toe aan de `trendsnapshots` tabel en gebruik ze als extra signal in `calculate_buzz_score()`.

4.5 Adaptieve Ingestie Uitbreiden

De `adaptive_ingest.py` detecteert al velocity anomalies en HN viral stories. Uitbreid met:[1]

- **Product Hunt nieuwe launches:** Check dagelijks PH API voor tools in de taxonomie met een featured of top_post status
- **ArXiv citation spikes:** Detecteer als een paper plots veel geciteerd wordt (cross-referentie met semantic_scholar API, gratis)
- **GitHub star velocity:** Als een repo >500 sterren per dag pakt, trigger targeted ingest

5. ToolStats Interface Uitbreidingsvoorstel

Om alle nieuwe data-punten in de Astro frontend bruikbaar te maken, breid `tools-domain.ts` uit:[2]

```
export interface ToolStats {
  // Bestaand
  slug: string
  buzz_score: number
  trend_direction: 'rising' | 'stable' | 'declining'
  mentions: number
  velocity: number
  source_count: number
  sources: string[]
  phase: TrendPhase
  days_active: number
  history: { date: string; mentions: number }[]

  // Nieuw — al berekend, nog niet geëxporteerd
```

```
acceleration: number      // 2e afgeleide velocity
expert_ratio: number      // 0.0–1.0 expert vs. mainstream
peak_mentions: number     // historische piek
first_seen: string | null  // ISO datum eerste mention
related_terms: string[]    // gerelateerde zoektermen

// Nieuw — nieuwe bronnen
youtube_mentions?: number // YouTube video count
npm_downloads?: number    // maandelijkse npm downloads
pypi_downloads?: number   // maandelijkse PyPI downloads
stackoverflow_questions?: number // SO vragen per maand

// Nieuw — LLM enrichment
narrative_summary?: string // NL duiding
forward_signal?: string     // NL voorspelling
}
```

6. Prioriteitenmatrix

Aanbeveling	Effort	Impact	Prioriteit
Exporteer bestaande berekende velden	Laag (30 min)	Hoog	*** Direct
Tool-specifieke enrichment automatiseren	Middel (1–2 dagen)	Hoog	*** Sprint 1
YouTube pipeline activeren	Middel (1 dag)	Middel	** Sprint 2
npm/PyPI download signals	Laag (2 uur)	Hoog voor dev-tools	*** Sprint 1
Product Hunt API (direct)	Middel (1 dag)	Middel	** Sprint 2
Stack Overflow API	Laag (1 uur)	Middel	** Sprint 2
ArXiv citation velocity	Hoog (2–3 dagen)	Hoog voor research tools	** Sprint 3
Discord activity scraper	Hoog	Laag (beperkte tools)	* Sprint 4

7. sync-tools-content.ts — ETL Gap Analyse

Het huidige ETL-script leest aitooldradar.json en genereert enrichmentdiff.json voor 7 velden:[2]

```
longDescription ← enrichment.description_long_nl
bestFor        ← enrichment.best_for_nl
```

```
useCases      ← enrichment.use_cases[]
strengths     ← enrichment.strengths[]
limitations   ← enrichment.limitations[]
pricing       ← enrichment.pricing_nl
openSource    ← enrichment.open_source
```

Gap: Het script vergelijkt wel of er diffs zijn, maar schrijft ze **nooit automatisch door** naar de tool-JSON-bestanden — alleen naar enrichmentdiff.json. Voeg een `--apply` flag toe om goedgekeurde diffs automatisch te mergen:

```
// In sync-tools-content.ts
if (args.includes('--apply')) {
  for (const diff of diffs) {
    const filePath = existingBySlug.get(diff.slug)!
    const current = await readJson<ToolContent>(filePath)
    ;(current as any)[diff.field] = diff.suggested
    await fs.writeFile(filePath, JSON.stringify(current, null, 2), 'utf8')
  }
  console.log(`Applied ${diffs.length} enrichment diffs.`)
}
```

Dit maakt de volledige loop van **NewsFlux** → **enrichment** → **Keystatic CMS** volledig geautomatiseerd.

References

1. [repomix-output.xml](#) - if i lentoenrich
time.sleepRATELIMITDELAY printf Enriched lenresults tools
errors errors if not args...
2. [AGENTS-2.md](#) - This file provides guidance to AI coding agents
WarpOz, Cursor, Claude, etc. when working in this re...